

Brief: Stripe payment flow

Stripe Payment Flow Brief

(Prepared for Jedidiah Shultz — Building Shultz / Solomon's Forge)

Executive Summary

- Stripe's Payment Intents API orchestrates card and alternative payment method (APM) transactions through a stateful, PCI-compliant flow that manages authentication, capture, and settlement in one lifecycle.^[^1]
 - Successful implementations hinge on a layered approach: frontend tokenization (Stripe.js + Elements), backend intent creation, webhook validation, fraud prevention (Radar), payout reconciliation, and clear customer-facing UX.
 - Stripe's modular stack (Billing, Connect, Invoicing, Terminal) enables expansion from simple one-time charges to subscription, marketplace, and point-of-sale experiences without re-architecting the core flow.
 - Recommended rollout: (1) sandbox integration (Week 1–2), (2) MVP checkout with card + Apple Pay (Week 3–4), (3) enable Radar rules + webhook hardening (Week 4–5), (4) analytics + automated payouts (Week 6).
 - Next steps: lock required API keys, configure webhooks, document failure-handling SOP, and stage migration path for IronEdit SaaS tiers and future marketplace offerings.
-

Table of Contents

1. [Stripe Ecosystem Overview](#)
 2. [End-to-End Payment Flow](#)
 3. [Key Components and Responsibilities](#)
 4. [Security, Compliance, and Risk Controls](#)
 5. [Operational Metrics and Monitoring](#)
 6. [Actionable Recommendations](#)
 7. [Timeline & Next Steps](#)
 8. [Appendix: Failure Handling Matrix](#)
-

Stripe Ecosystem Overview

Stripe provides an “economic infrastructure for the internet,” spanning collection, authorization, fraud mitigation, settlement, and reporting. Key modules relevant to Building Shultz and IronEdit:

Module	Purpose	Relevance
Payments / Payment Intents	Core API for accepting one-time and recurring payments	Required for IronEdit subscriptions, Builder's AI Blueprint upsells
Stripe Billing	Subscription lifecycle, invoices, proration, dunning	Turnkey management of IronEdit's \$19/\$29/\$59 tiers
Connect	Multi-party transfers (marketplaces)	Future marketplace of maker templates or service referrals
Radar	Machine learning fraud detection + custom rules	Safeguards card-not-present (CNP) sales
Sigma & Reporting	SQL-accessible data warehouse and dashboards	Revenue reconciliation and KPI tracking
Tax	Automated tax calculation/remittance	Critical when scaling digital goods globally
Terminal	In-person POS	Aligns with long-term brick-and-mortar ambitions

Stripe's global reach (135+ currencies, 50+ payment methods) reduces the need for multiple gateways. [^2] Their unified dashboard centralizes dispute management, payouts, and compliance audits.

End-to-End Payment Flow

1. Frontend Initialization

1. **Load Stripe.js + Elements** on the checkout page to ensure PCI compliance by tokenizing card data client-side; no sensitive card data touches our servers.[^3]
2. **Collect payment method details** via Elements (CardElement, PaymentElement) or native wallets (Apple Pay, Google Pay).
3. **Create Payment Method** (optional) client-side to reuse tokens for subscriptions or saved cards.

2. Backend Intent Creation

1. Server receives checkout request and authenticates customer/session.
2. **Payment Intent** is created via `POST /v1/payment_intents`, specifying amount, currency, capture method, metadata (order IDs), and accepted payment methods.
3. Stripe returns a client secret, which the frontend uses to confirm the payment without exposing the secret to malicious actors.

3. Customer Authentication

- Stripe automatically handles **3D Secure / Strong Customer Authentication (SCA)** challenges when required by card issuer or jurisdiction.
- The frontend calls `stripe.confirmCardPayment(clientSecret)`; if additional authentication is needed, Stripe modals prompt the user.

- Status transitions: `requires_payment_method` → `requires_confirmation` → `requires_action` → `processing` → `succeeded` (or `requires_capture` if manual capture enabled).

4. Confirmation & Capture

- **Automatic capture** (default): funds are captured immediately upon success.
- **Manual capture**: authorize now, capture later (up to 7 days for cards) via `POST /v1/payment_intents/{id}/capture` — useful for physical goods verification.
- On capture, Stripe transfers net funds (minus fees) to the pending balance.

5. Webhook Validation & Order Fulfillment

- Stripe sends events (e.g., `payment_intent.succeeded`, `payment_intent.payment_failed`) to configured webhook endpoints.
- Our server must verify signatures using `Stripe-Signature` header to prevent spoofing.
- Fulfillment logic (e.g., granting IronEdit license, emailing ebook) should rely on webhook confirmation, not purely frontend success, to avoid race conditions.

6. Settlement & Payouts

- Funds move from pending to available balance per payout schedule (daily/weekly).
- Automatic payouts deposit to connected bank accounts; manual payouts can be triggered via API or dashboard.
- For multi-currency, Stripe consolidates by currency balances; conversion fees apply when paying out in a different currency.

7. Post-Payment Events

- **Receipts** can be auto-emailed by Stripe or custom-sent via CRM.
- **Refunds**: initiated via dashboard or `POST /v1/refunds`, partial or full, referencing charge IDs.
- **Disputes**: Stripe notifies via webhook (`charge.dispute.created`); evidence must be submitted within 7–21 days depending on network.

Flow Diagram (Narrative)

Customer Checkout → Stripe.js tokenizes → Backend creates Payment Intent → Client confirms intent →

Key Components and Responsibilities

Component	Owner	Responsibilities	Notes
Frontend (React/Next/Vite)	Product Dev	Embed Stripe Elements, handle client secret, display errors	Keep <5 tabs per rule on PC agent when testing
Backend (Node/Electron helper API)	COO/Product	Securely store Stripe secret keys, create intents, verify webhooks	Use environment variables + restricted IAM

Component	Owner	Responsibilities	Notes
Stripe Dashboard	Finance/ Operations	Monitor payments, payouts, disputes, Radar alerts	Enable MFA, audit logins
Database	Engineering	Store customer profiles, subscription status, payment method IDs	Avoid storing raw PAN or CVV
Notification Layer	Marketing Ops	Email confirmations, Slack alerts, incident routing	Use webhook triggers for real-time updates
Accounting Stack	CPA role	Reconcile Stripe reports with QuickBooks / GnuCash	Export via Stripe Sigma or API

Security, Compliance, and Risk Controls

1. **PCI DSS:** Using Stripe Elements reduces scope to SAQ-A because Stripe hosts card data.[^4]
2. **Key Management:**
3. Use distinct restricted keys for development vs. production.
4. Rotate keys quarterly; log rotation event in ops runbook.
5. **Webhook Security:**
6. Validate signatures using Stripe's library.
7. Use unique webhook secret per environment.
8. **Radar & Fraud Controls:**
9. Enable adaptive machine learning rules.
10. Add custom rules (e.g., block high-risk BINs, velocity limits).
11. **Compliance Logs:**
12. Archive all API responses/events for 2 years for audit readiness.
13. Document incident response for disputes/refunds.
14. **Data Residency & Privacy:**
15. Honor GDPR/CCPA by referencing Stripe's data-processing terms; store localization preferences for customers.
16. **Availability & Redundancy:**
17. Stripe SLA is 99.9%; build retry logic with exponential backoff in case of transient errors.
18. Use idempotency keys for POST requests to prevent duplicate charges.

Operational Metrics and Monitoring

Metric	Target / Rationale	How to Measure
Authorization Success Rate	≥ 96%	Stripe Dashboard → Payments → Analytics
Chargeback Rate	< 0.8% of transactions	Radar reports + disputes tab
Refund Cycle Time	< 2 business days	Workflow automation logs

Metric	Target / Rationale	How to Measure
Average Ticket Value (ATV)	Track separately for \$19/\$29/\$59 tiers	Stripe Sigma query
Failed Payment Recovery	> 40% via dunning	Stripe Billing smart retries + email sequences
Payout Accuracy	Zero unreconciled payouts at month-end	Accounting reconciliation checklist
Radar False Positive Rate	< 3% of legitimate orders flagged	Review manual review queue weekly

Tie these metrics to quarterly OKRs; integrate with Slack via Stripe webhooks for real-time visibility.

Actionable Recommendations

- 1. Implement Payment Intents + Billing Immediately**
- Utilize Stripe Billing for IronEdit to automate recurring logic, proration, and dunning instead of building custom logic.
- 3. Expand Payment Methods Beyond Cards**
- Add Apple Pay, Google Pay, ACH debit, and buy-now-pay-later (Affirm/Klarna) to capture higher conversion, particularly for international makers.^[5]
- 5. Codify Webhook-Driven Fulfillment**
- Use AWS Lambda / DigitalOcean Functions as webhook processor to decouple from primary app.
- Queue fulfillment jobs (IronEdit license provisioning, SendGrid email) to guarantee delivery.
- 8. Deploy Radar Custom Rules**
- Block mismatched country/IP combos, velocity >3 failed attempts per hour, and disposable email domains.
- Schedule weekly manual review of flagged payments.
- 11. Automate Financial Ops**
- Enable Stripe's automated payouts (daily) but run a weekly reconciliation job exporting `balance_transactions`.
- Pipe data into accounting stack (QuickBooks/Sigma) for monthly close.
- 14. Prepare for Marketplace/Connect Expansion**
- Even if S&H Rentals is paused, design the integration so we can switch on Connect for future platform fees or revenue-sharing.
- 16. Document Failure Playbooks**

- 17. Build SOP for each failure code (insufficient_funds, card_declined, authentication_required).
- 18. Expose friendly error messaging in checkout and automated follow-up email with alternative payment instructions.

Timeline & Next Steps

Week	Milestone	Owner	Deliverables
Week 1	Sandbox Setup	COO/Product	- Create Stripe test account and restricted keys - Configure .env secrets storage - Spin up test webhook endpoint
Week 2	Checkout MVP	Engineering	- Implement Stripe Elements + Payment Intents - Unit tests for intent creation - Basic success/failure UI
Week 3	Subscription Flow	Product Dev	- Integrate Stripe Billing for tiered plans - Map plan IDs to IronEdit tiers - Draft dunning email templates
Week 4	Webhooks & Radar	COO	- Deploy webhook worker with signature validation - Configure Radar custom rules - Document incident response SOP
Week 5	Analytics & Payouts	Finance	- Schedule automated reports (Sigma/CSV) - Reconciliation checklist - Enable automatic payouts
Week 6	Launch Readiness	Exec	- End-to-end QA with 3D Secure scenarios - Load testing & monitoring setup - Go-live checklist + rollback plan

Immediate Next Steps (48 hours):

- 1. Secure Stripe API keys, store in password vault, and note in infrastructure log.
- 2. Provision webhook endpoint (DigitalOcean droplet or serverless) and whitelist Stripe IP ranges.
- 3. Draft PDF-ready documentation (this brief) and circulate to Jed + Tasia for visibility.
- 4. Kick off Week 1 tasks with status update to Jed (Telegram) per operating rules once implementation begins.

Appendix: Failure Handling Matrix

| Failure Scenario | Stripe Code / Event | Customer Message | Internal Action | SLA | | --- | --- | --- | --- | |
Card Declined | card_declined | "Your bank declined the transaction. Try another card or contact issuer." | Log attempt, increment velocity counter, notify Radar if repeated | Immediate | | Authentication Required | authentication_required / requires_action | "Additional verification required. Please complete the secure step." | Frontend triggers 3DS modal; monitor fallback email if user abandons | Real-time | | Insufficient Funds | insufficient_funds | "The payment couldn't be completed due to

insufficient funds.” | Offer ACH or alternative method via email | < 1 business day | | Duplicate Charge (network retry) | `idempotency_error` avoided via keys | N/A | Always send unique idempotency key; reconcile logs | Preventive | | Webhook Signature Failure | 400 error | Customer unaffected | Alert Slack channel, investigate secret mismatch | < 2 hours | | Dispute / Chargeback | `charge.dispute.created` | “We received a dispute; we’ll keep you updated.” | Submit evidence (proof of service, logs) via Dashboard | 5 business days | | Refund Requested | `charge.refunded` | “Your refund is being processed (3–10 days).” | Track refund ledger, update license access | Same day |

References

[^1]: Stripe Docs – “Payment Intents” <https://stripe.com/docs/payments/payment-intents>

[^2]: Stripe Global Coverage <https://stripe.com/global>

[^3]: Stripe.js & Elements Security <https://stripe.com/docs/js>

[^4]: PCI DSS & Stripe <https://stripe.com/docs/security/guide>

[^5]: Supported Payment Methods <https://stripe.com/payments/payment-methods>

Prepared by: Sol — COO, Solomon's Forge

Date: 2026-05-23